

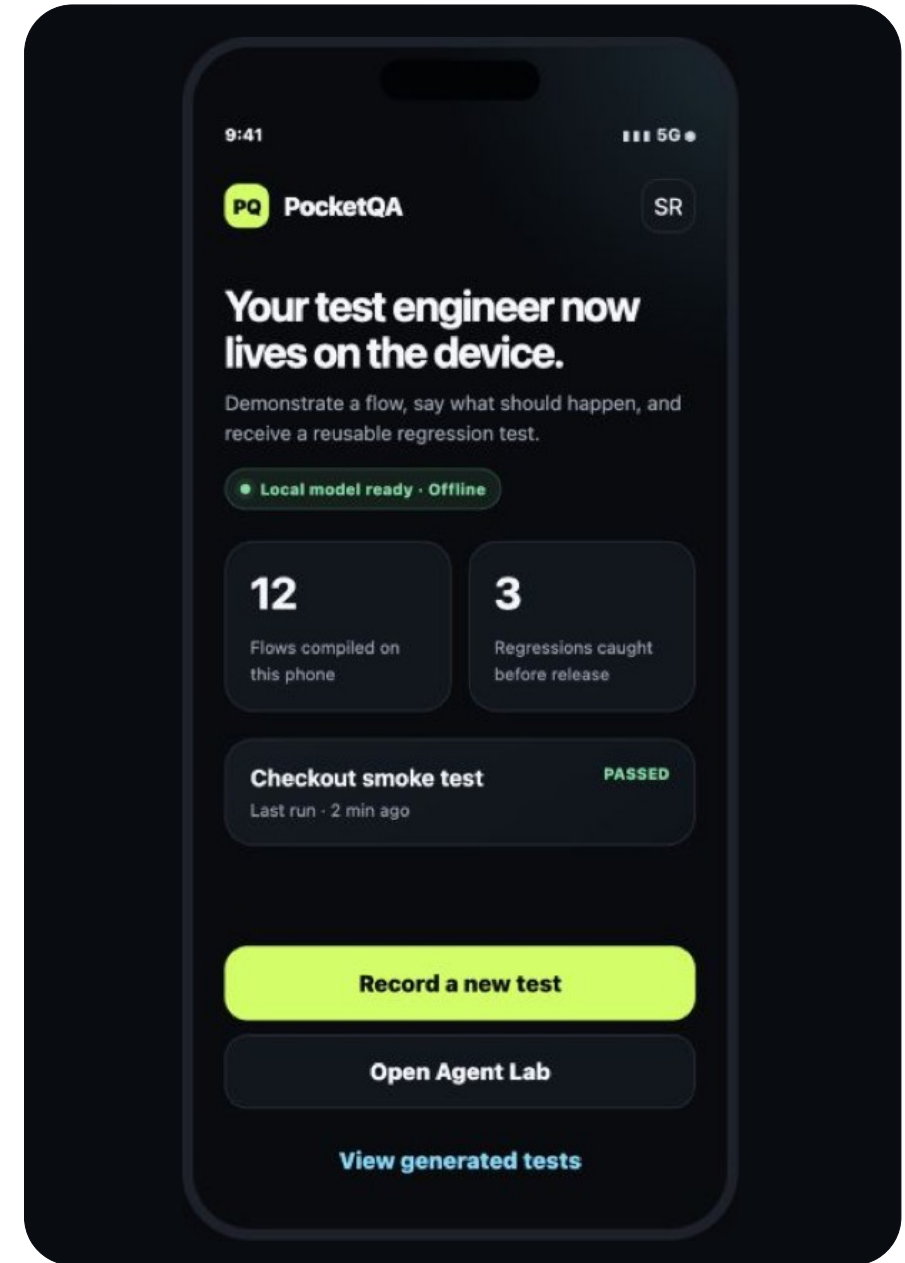
# PocketQA

## Show a flow once. Ship the regression test.

A private, on-device AI test author—and safe QA agent—for mobile teams.

ON-DEVICE AI · MOBILE QA

Bengaluru City Battle · 29–30 Aug 2026



# Mobile teams know what to test. Authoring it is the bottleneck.

**“Make checkout work” is clear to a human—but not executable by CI.**

Small teams repeatedly choose between manual QA, brittle scripts, or cloud tools that do not understand the developer’s specific intent.

---

01

## Test intent stays trapped

Expected behaviour lives in tickets, voice notes and the developer’s head.

---

02

## Automation takes specialist time

Selectors, waits and assertions must be authored and maintained manually.

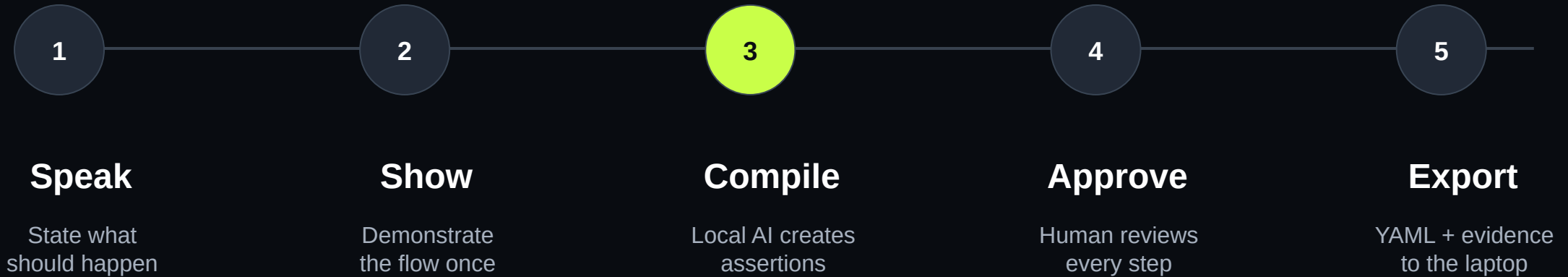
---

03

## Failures arrive without context

A red CI run rarely contains the human intent and exact evidence together.

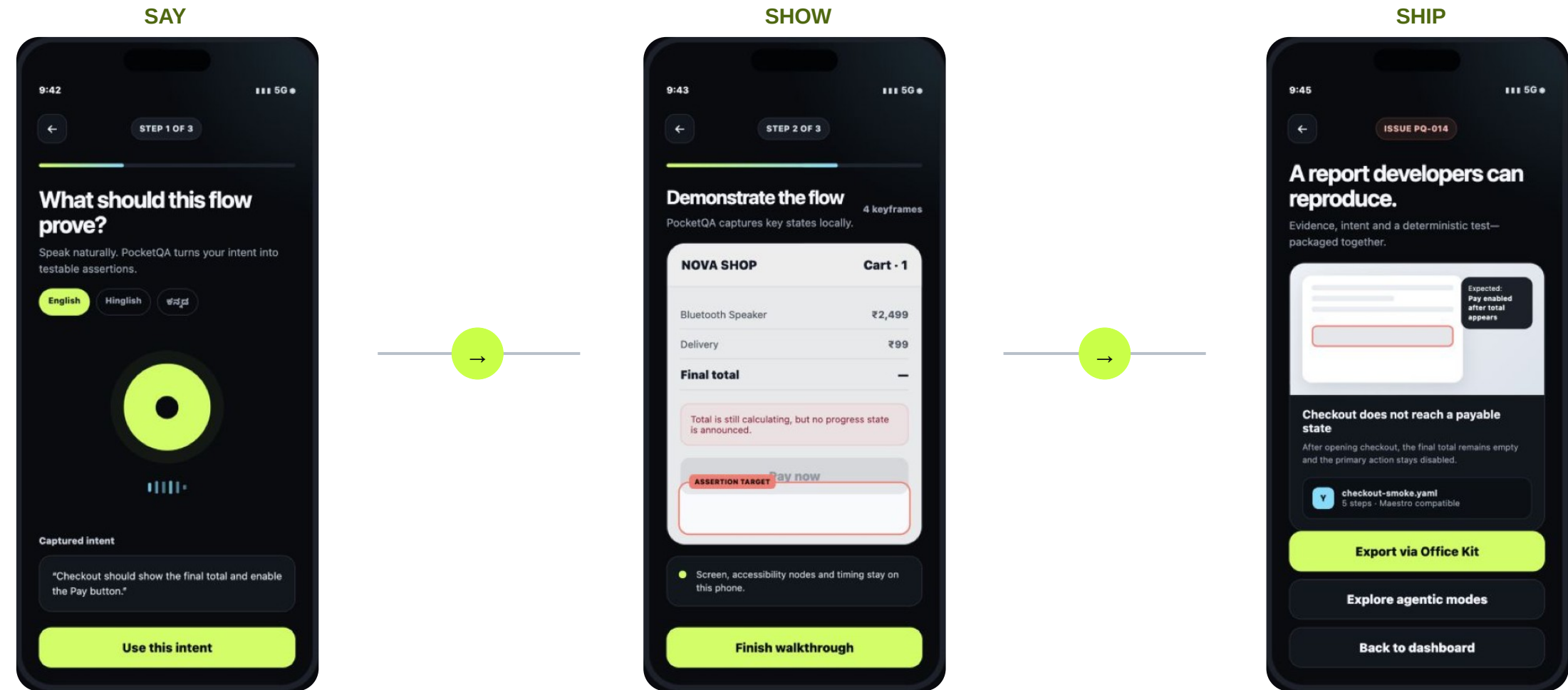
# PocketQA turns intent into a deterministic test in five moves.



THE RESULT

**One human walkthrough becomes a reviewable regression test, reproducible bug report and portable CI asset.**

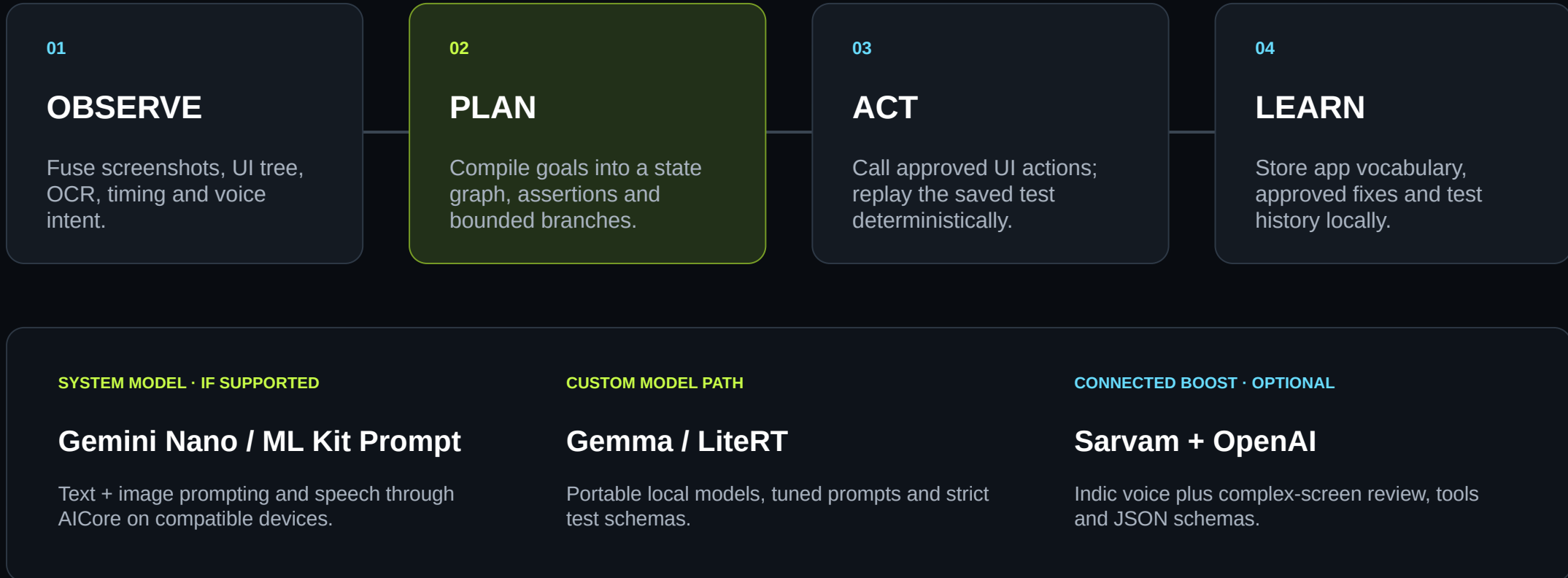
# The core demo remains simple enough to trust.



# PocketQA authors the test; existing tools can still run it.

|               | MAESTRO<br>Test execution framework | FIREBASE ROBO<br>Automated cloud exploration | POCKETQA<br>On-device semantic authoring |
|---------------|-------------------------------------|----------------------------------------------|------------------------------------------|
| INPUT         | Developer-authored flow             | APK + generic crawl                          | Human demo + spoken intent               |
| UNDERSTANDING | Commands and selectors              | UI hierarchy exploration                     | Expected outcome + screen state          |
| PROCESSING    | Runner on device/emulator           | Cloud test infrastructure                    | Local model on target phone              |
| OUTPUT        | Pass/fail + artifacts               | Crawl graph + logs/video                     | Approved test + bug evidence             |
| RELATIONSHIP  | PocketQA can export to it           | Different coverage mode                      | The authoring layer                      |

# A local agent can observe, plan, act and learn—inside explicit rails.



**POLICY:** the model proposes · the executor validates · guarded actions stop · every result leaves evidence

# One trusted loop can support six specialist agents.

## BUILD NEXT

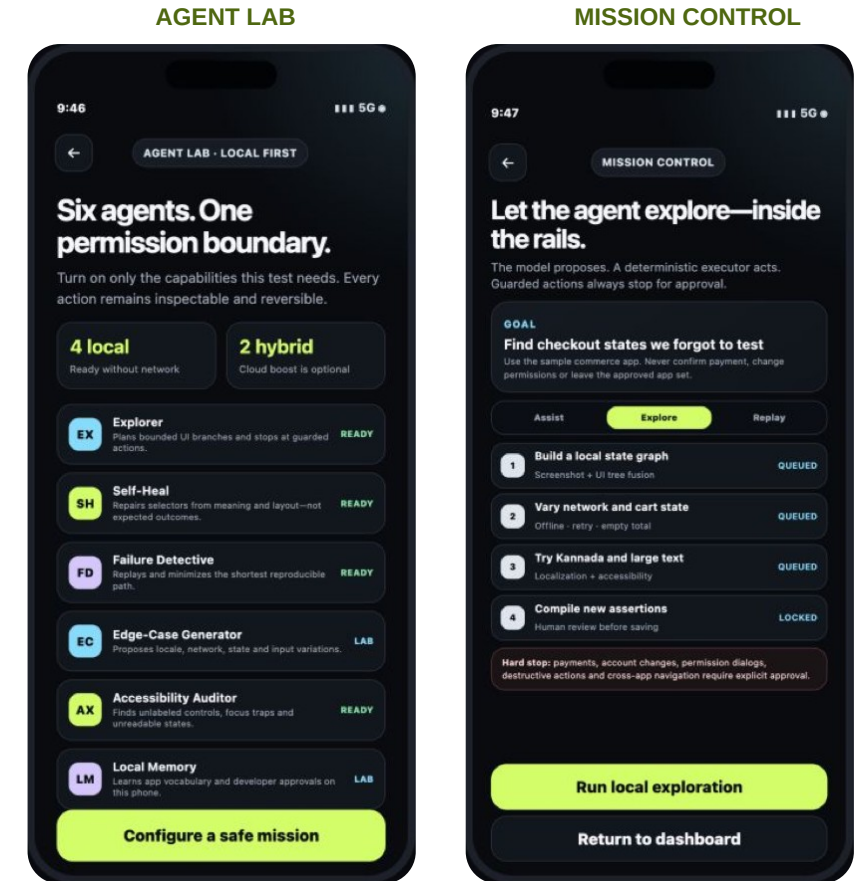
Explorer — bounded state graph + branch search  
 Self-Heal — semantic selectors, never assertions  
 Failure Detective — replay, minimize and explain

## SECOND WAVE

Edge-Case Generator — locale, network, state and input variants  
 Accessibility Auditor — labels, focus, large text and contrast  
 Local Memory — app vocabulary, approvals and test history

## PLATFORM BETS

Android AppFunctions as local tools · OEM Computer Control · multi-device cooperative test agents



Each agent is opt-in. High-risk actions remain outside the autonomous tool set.

# Finish the trusted loop before adding autonomy.

GATE 1 · MUST WORK

## Authoring loop



React Native shell  
Screen + UI-tree capture  
Intent → strict JSON  
Human approval + YAML

GATE 2 · WINNING WOW

## One safe Explorer



Build a local state graph  
Try one bounded branch  
Stop before guarded action  
Show novel-state evidence

GATE 3 · ONLY IF STABLE

## Specialist agents



Selector self-heal  
Failure minimizer  
Accessibility audit  
Indic connected voice

DEMO RULE

Agent mode stays an opt-in tab until the five-step authoring flow passes end to end.



# Start with test authoring. Grow into a private QA agent.

- 01 **Human intent in**  
Voice, demonstration and product context.
- 02 **Bounded local agents explore**  
Visible plan, approved tools and hard stops.
- 03 **Deterministic evidence out**  
Reviewable assertions, replay and portable tests.

